_VOIS RAG

User guide and system documentation

A practical guide to starting the local application, adding documents, asking questions in English or Arabic, and understanding how answers are produced.



LOCAL APP	DOCUMENTS	MODELS
Gradio at 127.0.0.1:7860	PDF, DOCX, PPTX; up to 5 ready docs	Qwen2.5:3b, Nomic, BGE-M3
Ollama at 127.0.0.1:11435	Project files and indexes on E:	English and Arabic workflows

This guide reflects the project source and configuration inspected on 15 October 2026.

Start and use the app

The Windows launcher starts the project-managed Ollama service and the Gradio interface. Keep the project on E: so the configured models, Python environment, index, and project data stay together.

STEP	ACTION	WHAT TO DO
	First-time setup	Run run_rag_one_click.bat once. It prepares the project environment and required local models.
2	Start	Run start_rag.bat. Wait for Gradio to report its local URL.
	Open	Visit http://127.0.0.1:7860 in a browser on the same computer.
4	Choose or add a source	Select a document in the sidebar, or use Add document to upload a supported file.
	Ask	Type a question in the bottom message field and press Enter or Send. Answers include source pages or slides when available.

Using the interface

- Document picker: switches the source used for new questions. A chat belongs to its document.
- New chat: starts a separate conversation for the selected document.
- Chat filter: type to find an existing conversation; rename, reorder, or delete it from the chat controls.
- Rename/delete document: expand the corresponding sidebar section. Document deletion is permanent for that source/index; the UI offers a choice about chat history.
- Translate button: changes the display language and translates the visible answer; it does not change the document or its index.

Question tips

Ask one specific question at a time. Include names, dates, or terms exactly as they appear in the source. Check the cited page for important decisions. If the answer says the document does not contain the fact, try a more specific wording or verify that the source was indexed correctly.

Documents and indexing

A successful upload is extracted, split into language-aware chunks, embedded locally, and added to a dedicated Chroma collection. The app records the document only after indexing completes; failures trigger cleanup of partial records and files.

Supported files and limits

FORMAT	EXTRACTION	NOTES
PDF	Page text, page-number markers	Scanned pages need optional Tesseract OCR; OCR is off by default.
DOCX	Paragraph text	Password-protected Office files are not supported.
PPTX	Slide text, slide-number markers	Keep slide text selectable for best extraction.

Current ingestion safeguards

- At most five ready documents are kept in the library.
- The default per-file limit is 40 MB; extracted text is bounded to 8 million characters.
- PDF page count is limited to 3,000. Office archive expansion and entry counts are bounded.
- English documents use nomic-embed-text; Arabic documents use multilingual bge-m3. Query embeddings must match the document's model.
- English chunks use a 1,500-character target with 100 overlap. Arabic chunks use a 1,000-character target with 50 overlap.
- Text extraction, chunk embedding, and indexing are local. Large documents can take time, especially on CPU.

For scanned PDFs

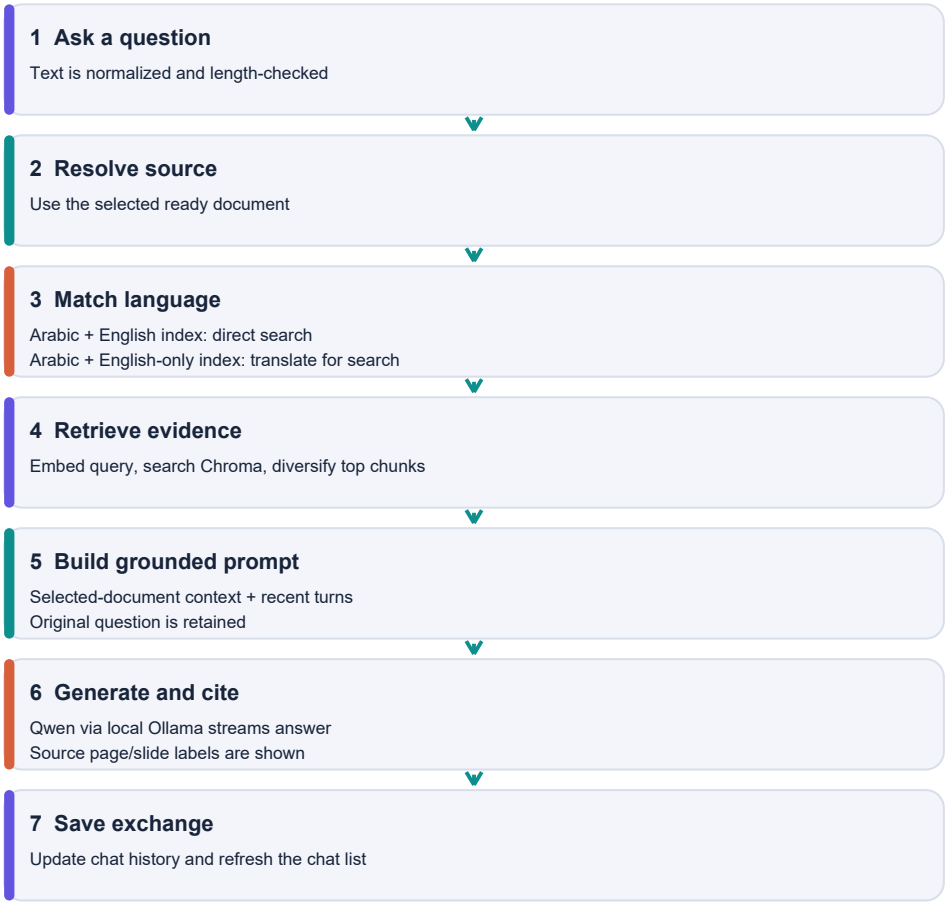
Install Tesseract with the required language data, then set RAG_OCR=1 and RAG_OCR_LANG=eng+ara in the same terminal before launching start_rag.bat. Use only language codes actually installed. OCR is attempted only for pages with little or no extractable text and can be substantially slower.

Where project data lives

PATH	PURPOSE
.tools\ollama-models	Ollama model files
.tools\uploads	Copies of uploaded source documents
.tools\documents.json	Document registry and embedding/collection metadata
.tools\chat_history.json	Chat titles, ordering, timestamps, and message arrays
chroma_db	Persistent vector collections and chunk metadata

How a question is answered

The app resolves the chosen document, retrieves evidence from its collection, builds a bounded prompt, and streams a local model response. The original question is passed to answer generation even when a translated search query is used for retrieval.



If retrieval fails, the app falls back to similarity search.

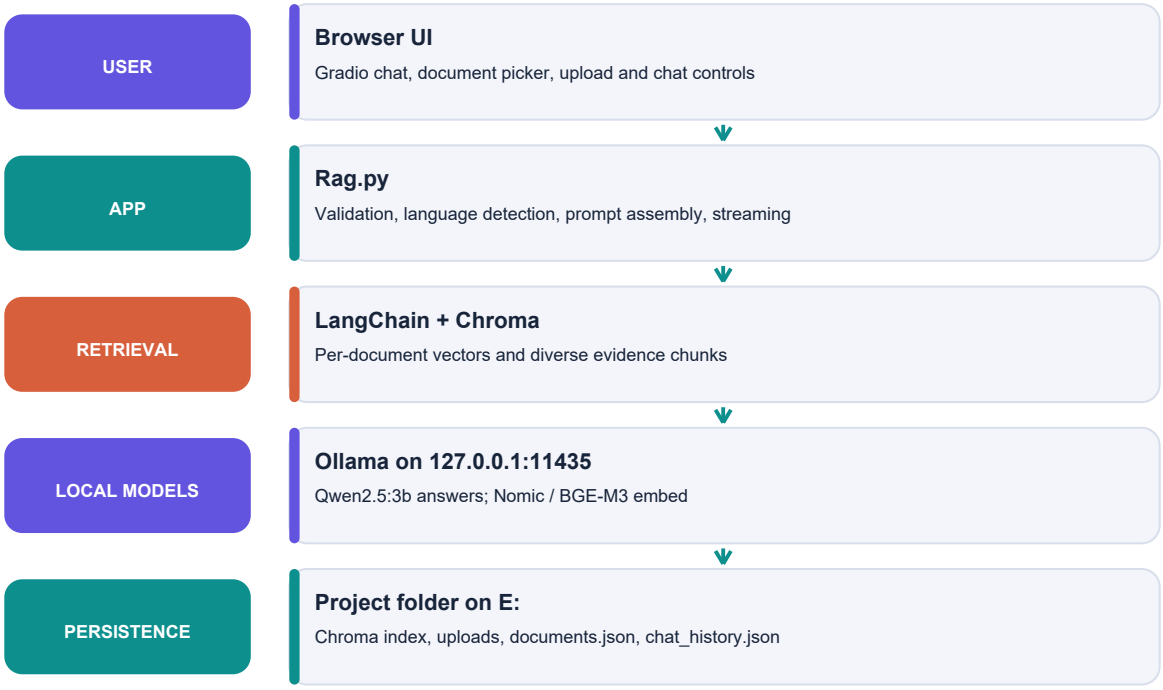
Retrieval settings

SETTING	DEFAULT	EFFECT
RAG_TOP_K	5	Maximum evidence chunks returned for the answer.
RAG_RETRIEVAL_DIVERSITY	0.75	MMR diversity; lower values favor similar top matches, higher values favor variety.
RAG_CTX	6144	Model context size used by the answer and search-translation clients.
RAG_KEEP_ALIVE	10m	Keep the local model loaded after a request to reduce reload delays.

A first Arabic query against a document indexed with the English-only embedding model may need one translation call before retrieval. This can add latency. Arabic-indexed documents use BGE-M3 and can be searched directly across supported languages.

System architecture

The application and model service communicate over loopback. The user interface is Gradio, the orchestration is in Rag.py and ingest.py, vector search is in Chroma, and Ollama runs the local chat and embedding models.



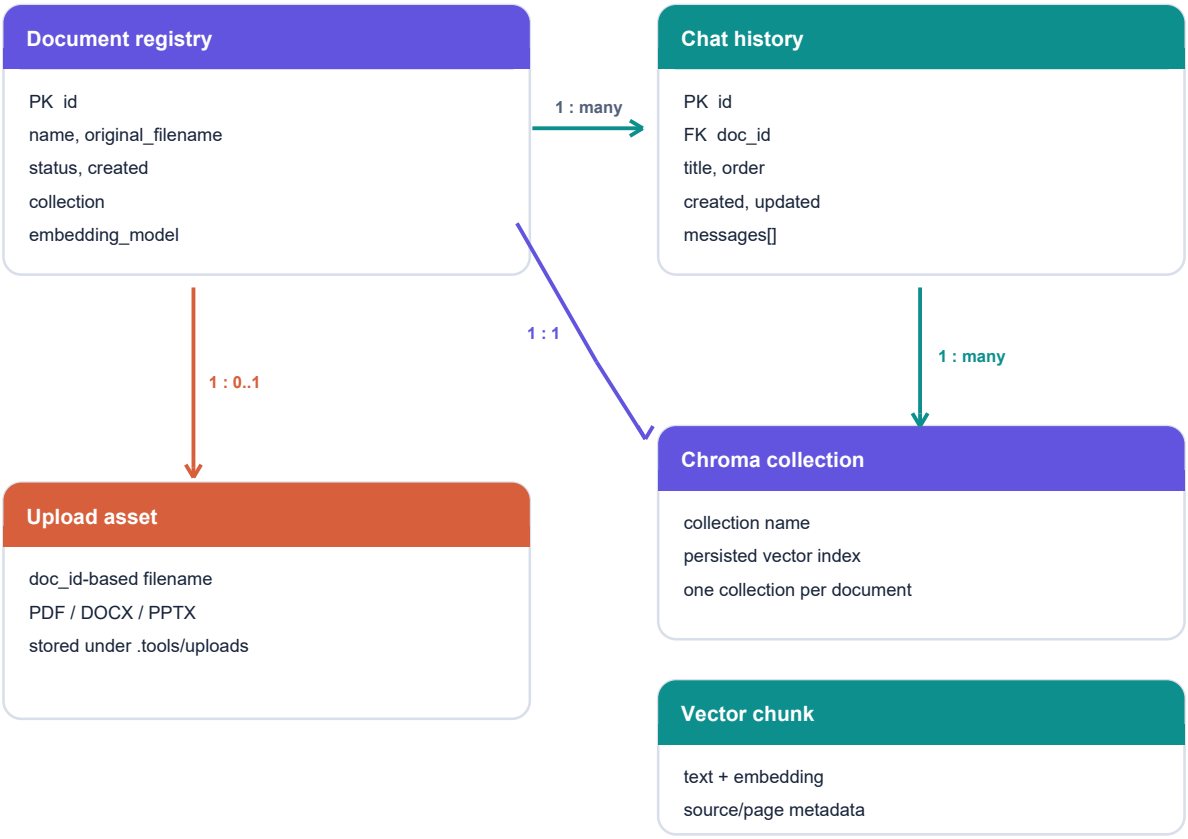
All application/model services are local; the app binds to loopback by default.

Runtime components

COMPONENT	ROLE
Gradio	Browser interface for upload, document/chat selection, questions, streaming answers, and citations.
Rag.py	Coordinates validation, retrieval, prompts, model calls, chat persistence, and UI actions.
ingest.py	Validates PDF/DOCX/PPTX, extracts text, adds page/slide markers, chunks, and indexes.
Ollama	Serves Qwen2.5:3b for answers, nomic-embed-text for English documents, and BGE-M3 for Arabic documents.
Chroma + JSON	Chroma stores vectors/chunks. JSON files store document and conversation records.

Conceptual entity relationship diagram

This is a logical relationship view, not a relational schema. The project uses JSON registry/history files and Chroma collections. Chat messages are nested within a chat record, and document-to-index links use a collection name.



Key relationships

- One document can have many chats. Each chat has a document identifier and an ordered messages array.
- One document record points to one Chroma collection; the collection contains many chunk vectors with source/page or slide metadata.
- An uploaded source copy is stored under .tools/uploads using the generated document id in its filename.
- Deleting a document removes its Chroma collection and stored upload. Chat history can be retained in view-only form when selected in the UI.

Registry safety

documents.json and chat_history.json are written atomically by writing a temporary file and replacing the target. Back up the whole project data set together so registry entries, source files, and indexes remain aligned.

Troubleshooting and safe operation

SYMPTOM	CHECK	NEXT STEP
Browser cannot open the app	Check Gradio startup output and port 7860.	Run start_rag.bat. If port 7860 is occupied, close the other app instance or set RAG_PORT to another local port.
Ollama connection error	Project service should answer on 127.0.0.1:11435.	Use start_rag.bat, which starts the project-managed Ollama binary and verifies availability.
Model not found	Required tags are qwen2.5:3b, nomic-embed-text, and bge-m3.	Run the one-click installer or pull the exact missing model into the project Ollama instance.
Upload failed or no text found	Check file type, size, corruption, password, and whether text is selectable.	For scanned PDFs, configure Tesseract/OCR. Re-upload after fixing the source.
Weak or incomplete answer	Open cited pages; check extraction and document selection.	Ask a narrower question. The model can be wrong even when citations are present.
Slow first answer	Local model may be loading; CPU inference is hardware-bound.	Model keep-alive reduces later reloads. Cross-language search translation can add a call on first use.

Security and privacy

• Default binding is 127.0.0.1. Do not expose the app to a LAN or the internet without credentials and a TLS-protected reverse proxy.

• Uploaded files, chat history, settings, and Chroma indexes are local but are not encrypted by this application. Protect the Windows account and project folder; use BitLocker where available.

• Treat document contents as untrusted. Prompt rules help resist document-injected instructions but cannot guarantee the model will ignore them.

• Validate important answers against the cited source. A citation is evidence location, not proof the generated summary is correct.

• Keep backups of .tools/uploads, .tools/documents.json, .tools/chat_history.json, and chroma_db together.

Project files

Rag.py, ingest.py, docmanager.py, cleanup_registry.py, start_rag.bat, run_rag_one_click.bat, requirements.txt, and README.md.